Proposal: The Rabta Programming Language

A Concurrent, Message-Passing Language with Compile-Time Checked Conversations

Course: Compiler Construction (7th Semester) | Pipeline: Flex -> Bison (LALR) -> LLVM IR -> AOT / JIT

Ansa Mubasher| Ahmed Yasin| Rana Muhammad Abdullah

1. Purpose of the Language

Concurrent programs built on threads, locks and ad-hoc message queues are hard to get right. A very common bug is a *protocol mismatch*: one thread waits for a message that the other never sends, or sends the wrong type of data, causing deadlocks or crashes that only appear at run time.

Rabta targets **concurrent and message-passing programming**. A program is a set of independent actors (*bande*, "people") that talk to each other only through typed channels, just like people in a *rabta* (connection). The central idea is that the structure of each chat is written down as a **conversation protocol**, and the compiler verifies at compile time that every actor follows it. Programs that would deadlock or exchange mismatched data are rejected before they run. The language keeps a small colloquial, subcontinental keyword set (`shuru`/`khatam`, `agar`/`warna`, `jab_tak ... chal`) so that code reads like spoken reasoning, but the novelty is in the concurrency model, not the spelling of keywords.

2. Key Features

- **Conversation protocols (`baatcheet`).** A protocol declares the exact order, direction and type of messages in a chat. Example: the client sends an i64, then the server replies with an i64. This is a simplified form of *session types*, checked by a semantic pass over the AST.
- **Typed channel endpoints.** `nali ch: P` creates a channel for protocol P and gives two endpoints (`ch.client`, `ch.server`). Each endpoint carries its own protocol state, so `suno` (receive) before `bolo` (send), a wrong message type, or an unfinished chat is a compile-time error.
- **Actors (`banda`) and spawning (`paida`).** Actors have private state and share nothing; they run as lightweight threads. No locks or shared variables are exposed to the programmer, so data races are impossible by construction.
- **Choice and group chats (planned extensions).** `chuno` lets one side pick between labelled branches (the other side must handle every branch), and `mehfil` broadcasts one message to a group of actors. A parallel range loop `saath ghoom [i : 0 -> n]` splits iterations across actors.
- **Sequential core.** Statically typed i64 values (`rakho`), bounded arrays (`thali`), functions (`kaam`/`bulao`/`waapis`), conditionals and loops, so ordinary algorithms such as sorting and recursion are still expressible inside each actor.

3. Compiler Design (how the requirements are met)

Stage	Role in Rabta
Flex	Tokenizes keywords (`baatcheet`, `banda`, `nali`, `bolo`, `suno`, `paida`, ...), identifiers, integers and comments.
Bison (LALR(1))	Unambiguous grammar for protocols, actors, functions, statements and expressions; builds the AST.
Semantic pass	Type checking plus **protocol checking**: simulates each channel endpoint's state machine over the actor body.
LLVM IR builder	Functions become `llvm::Function`; arrays use GEP; conditions use `br i1`. Each actor lowers to a function run by a small C runtime (thread + bounded queue); `bolo`/`suno` lower to calls to runtime send/recv.
Backend	JIT (LLVM ORC) for quick testing and AOT object/executable output; both use the same

	IR.

4. Small Example

A client asks a server actor for the sum of 1..n. The protocol `Hisaab` fixes the conversation, and the compiler checks both sides against it.

```
// Protocol: who speaks, in what order, with what type
baatcheet Hisaab shuru
    client bolega i64      // n
    server bolega i64      // sum of 1..n
khatam

banda server(sira s : Hisaab.server) shuru
    rakho n = suno s       // receive n
    rakho total = 0
    ghoom [i : 1 -> n + 1] chal
        total = total + i
    khatam
    bolo s, total          // send the answer; protocol now complete
khatam

kaam main() shuru
    nali line : Hisaab     // creates line.client and line.server
    paida server(line.server)      // spawn the server actor
    bolo line.client, 100          // send n
    rakho ans = suno line.client   // receive the answer
    dikhao(ans)                    // prints 5050
    waapis 0
khatam
```

Rejected at compile time. If `main` called `suno line.client` before `bolo line.client, 100`, both actors would wait forever. Rabta reports: *protocol violation: Hisaab.client must speak first (expected bolo i64, found suno)*.

```
kaam main() shuru
    nali line : Hisaab
    paida server(line.server)
    rakho ans = suno line.client   // ERROR: client must bolo first
    waapis 0
khatam
```

Originality. Rabta is not a re-skinned C. Its distinguishing contribution is a conversation-first concurrency model whose protocols are verified by the compiler and lowered to LLVM IR plus a minimal runtime.